

Sprint 9 Structured Notes: Callbacks and Higher-Order Array Methods

0. What Sprint 9 Is About

Sprint 9 is about processing arrays with functions.

Before this sprint, you mostly used loops to go through arrays:

```
const numbers = [1, 2, 3];

for (const number of numbers) {
  console.log(number);
}
```

In Sprint 9, you learn another way:

```
const numbers = [1, 2, 3];

numbers.forEach(number => {
  console.log(number);
});
```

The main idea is:

Functions can be treated like values. You can pass a function into another function.

This lets you use array methods like:

- `forEach()`
- `map()`
- `filter()`
- `reduce()`
- `sort()`
- `every()`
- `some()`

Main goal:

Use functions as values to process collections clearly.

By the end of Sprint 9, you should understand:

- callbacks
- higher-order functions
- function factories
- `forEach()`
- `map()`
- `filter()`
- `reduce()`
- accumulators
- method chaining
- `sort()`
- compare functions
- mutation with `sort()`
- copying before sorting
- `every()`
- `some()`
- arrow function returns
- the difference between side effects and returned values
- common beginner mistakes with array methods

1. Current Progress Before Sprint 9

Sprint 9 depends on the earlier sprints.

Earlier Sprint	What You Learned	How It Helps With Sprint 9
Sprint 1	JavaScript can respond to actions on a web page.	Array methods can later help process data before updating the page.
Sprint 2	Values, data types, <code>let</code> , <code>const</code> , and references.	Arrays and functions are values. You can store and pass them around.
Sprint 3	Strings, indexing, and methods.	Array methods also use method syntax like <code>.map()</code> and <code>.filter()</code> .
Sprint 4	Numbers, booleans, comparisons, and conditions.	<code>filter()</code> , <code>every()</code> , and <code>some()</code> use boolean tests.
Sprint 5	Functions, parameters, arguments, returns, scope, callbacks, and closures.	Sprint 9 uses functions as callback arguments.
Sprint 6	Arrays, indexes, array methods, shallow copies.	Sprint 9 adds more powerful array-processing methods.

Earlier Sprint	What You Learned	How It Helps With Sprint 9
Sprint 7	Loops and iteration.	Sprint 9 gives cleaner alternatives to many loop patterns.
Sprint 8	Objects, nested data, arrays of objects, methods.	Sprint 9 often processes arrays of objects.

Simple summary:

Sprint 7 taught you how to manually loop through arrays. Sprint 9 teaches you how to process arrays by giving functions to array methods.

2. The Core Idea of Sprint 9

JavaScript functions are values.

That means you can:

- store a function in a variable
- pass a function into another function
- return a function from another function
- use functions inside array methods

Example:

```
function sayHello() {
  console.log("Hello");
}

const task = sayHello;

task();
```

Expected output:

```
Hello
```

This works because `sayHello` is a function value.

Important:

```
const task = sayHello;
```

This stores the function itself.

But this:

```
const task = sayHello();
```

runs the function immediately and stores its return value.

That difference matters a lot in Sprint 9.

3. What Is a Callback?

A callback is a function passed into another function.

Simple meaning:

A callback is a function you give to another function so it can run later.

Example:

```
function runTask(callback) {  
  callback();  
}  
  
function sayHello() {  
  console.log("Hello");  
}  
  
runTask(sayHello);
```

Expected output:

```
Hello
```

Here:

```
sayHello
```

is the callback.

It is passed into:

```
runTask
```

Then `runTask` calls it here:

```
callback();
```

4. Passing a Function vs Calling a Function

This is one of the most important beginner details.

Passing a function

```
runTask(sayHello);
```

This means:

Give the `sayHello` function to `runTask`.

The function is not called immediately by this line.

Calling a function

```
runTask(sayHello());
```

This means:

Run `sayHello()` now, then pass its returned value into `runTask`.

That is usually not what you want when a callback is expected.

Example:

```
function sayHello() {  
  console.log("Hello");  
}
```

```
function runTask(callback) {  
  callback();  
}  
  
runTask(sayHello());
```

This causes a problem because `sayHello()` returns `undefined`.

So the code becomes similar to:

```
runTask(undefined);
```

Then this fails:

```
callback();
```

because `undefined` is not a function.

Beginner rule:

When passing a callback, usually pass the function name without parentheses.

Correct:

```
runTask(sayHello);
```

Usually wrong:

```
runTask(sayHello());
```

5. What Is a Higher-Order Function?

A higher-order function is a function that does at least one of these:

1. Takes another function as an argument.
2. Returns another function.

Example that takes a function:

```
function runTask(callback) {  
  callback();  
}
```

`runTask` is a higher-order function because it takes `callback`, which is a function.

Example that returns a function:

```
function createGreeting() {  
  return function() {  
    return "Hello";  
  };  
}  
  
const greeting = createGreeting();  
console.log(greeting());
```

Expected output:

Hello

Array methods like these are higher-order methods:

```
forEach()  
map()  
filter()  
reduce()  
sort()  
every()  
some()
```

Why?

Because they receive callback functions.

6. Function Factory Review

A function factory is a function that creates and returns another function.

Example:

```
function multiplyBy(factor) {  
  return function(number) {  
    return number * factor;  
  };  
}  
  
const double = multiplyBy(2);  
const triple = multiplyBy(3);  
  
console.log(double(5));  
console.log(triple(5));
```

Expected output:

```
10  
15
```

Step-by-step:

```
const double = multiplyBy(2);
```

This creates a new function that remembers `factor = 2`.

Then:

```
console.log(double(5));
```

means:

```
5 * 2
```

So the result is:

```
10
```

This connects to Sprint 9 because it shows that functions can be returned and reused as values.

7. Array Callback Shape

Most Sprint 9 array methods use callbacks with this general shape:

```
array.method((item, index, array) => {  
  // code here  
});
```

The callback can receive:

Parameter	Meaning
item	The current array item.
index	The current item's index.
array	The whole original array.

Example:

```
const fruits = ["apple", "banana", "cherry"];  
  
fruits.forEach((fruit, index, wholeArray) => {  
  console.log(fruit);  
  console.log(index);  
  console.log(wholeArray);  
});
```

You do not always need all three parameters.

Most beginner code only needs the first one:

```
fruits.forEach(fruit => {  
  console.log(fruit);  
});
```

Sometimes you need the index:

```
fruits.forEach((fruit, index) => {  
  console.log(index, fruit);  
});
```

Beginner rule:

Start with the current item. Add `index` only when you need it.

8. Arrow Functions in Sprint 9

Sprint 9 often uses arrow functions because callbacks are usually short.

Normal function:

```
function double(number) {  
  return number * 2;  
}
```

Arrow function:

```
const double = number => number * 2;
```

Both can produce the same result:

```
console.log(double(5));
```

Expected output:

```
10
```

Arrow function with implicit return

```
number => number * 2
```

This automatically returns `number * 2`.

Arrow function with explicit return

```
number => {  
  return number * 2;  
}
```

When you use curly braces `{}`, you usually need `return`.

Important beginner rule:

No curly braces can mean implicit return. Curly braces require explicit `return` if you want to send back a value.

9. `forEach()`

Use `forEach()` when you want to do something once for each array item.

Simple meaning:

For each item in the array, run this function.

Example:

```
const fruits = ["apple", "banana", "cherry"];

fruits.forEach(function(fruit) {
  console.log(fruit);
});
```

Expected output:

```
apple
banana
cherry
```

Same example with an arrow function:

```
const fruits = ["apple", "banana", "cherry"];

fruits.forEach(fruit => {
  console.log(fruit);
});
```

Expected output:

```
apple
banana
cherry
```

10. `forEach()` With Index

The callback can also receive the index.

Example:

```
const fruits = ["apple", "banana", "cherry"];

fruits.forEach((fruit, index) => {
  console.log(index, fruit);
});
```

Expected output:

```
0 apple
1 banana
2 cherry
```

Here:

```
fruit
```

is the current item.

```
index
```

is the current index.

This is useful when you need numbering or position information.

11. What `forEach()` Returns

`forEach()` returns `undefined`.

Example:

```
const numbers = [1, 2, 3];
```

```
const result = numbers.forEach(number => {  
  return number * 2;  
});  
  
console.log(result);
```

Expected output:

```
undefined
```

Even though the callback has `return`, `forEach()` does not collect the returned values.

Beginner rule:

Use `forEach()` for side effects, not for creating a new array.

Side effects include:

- printing with `console.log()`
- changing page text
- adding elements to the DOM
- updating an external variable
- showing an alert
- saving data

Example side effect:

```
const names = ["Ada", "Grace", "Linus"];  
  
names.forEach(name => {  
  console.log(`Hello, ${name}`);  
});
```

Expected output:

```
Hello, Ada  
Hello, Grace  
Hello, Linus
```

12. `forEach()` vs `for...of`

These two examples do similar work.

Using `for...of`:

```
const fruits = ["apple", "banana", "cherry"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

Using `forEach()`:

```
const fruits = ["apple", "banana", "cherry"];

fruits.forEach(fruit => {
  console.log(fruit);
});
```

Both print:

```
apple
banana
cherry
```

Simple comparison:

Tool	Use When
<code>for...of</code>	You want a simple loop that can use <code>break</code> or <code>continue</code> .
<code>forEach()</code>	You want to run a callback once for every item.

Important:

You cannot use `break` or `continue` inside `forEach()` the same way you do inside normal loops.

If you need to stop early, a normal loop is often clearer.

13. `map()`

Use `map()` when you want to transform every item and create a new array.

Simple meaning:

Take each item, change it, and put the changed value into a new array.

Example:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => {
  return number * 2;
});

console.log(doubled);
```

Expected output:

```
[2, 4, 6]
```

Shorter version:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => number * 2);

console.log(doubled);
```

Expected output:

```
[2, 4, 6]
```

14. What `map()` Returns

`map()` returns a new array.

Example:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => number * 2);

console.log(numbers);
console.log(doubled);
```

Expected output:

```
[1, 2, 3]
[2, 4, 6]
```

The original array is unchanged.

The new array contains the transformed values.

Beginner rule:

Use `map()` when the output should have the same number of items as the input.

Example:

Input:

```
[1, 2, 3]
```

Output:

```
[2, 4, 6]
```

Same number of items: 3 in, 3 out.

15. `map()` With Objects

`map()` is very useful with arrays of objects.

Example:

```
const students = [
  { name: "Ada", score: 90 },
```



```
    { name: "Grace", score: 85 },
    { name: "Linus", score: 70 }
  ];

  const names = students.map(student => student.name);

  console.log(names);
```

Expected output:

```
["Ada", "Grace", "Linus"]
```

What happened?

Each student object was transformed into only the student's name.

Input:

```
[
  { name: "Ada", score: 90 },
  { name: "Grace", score: 85 },
  { name: "Linus", score: 70 }
]
```

Output:

```
["Ada", "Grace", "Linus"]
```

Another example:

```
const students = [
  { name: "Ada", score: 90 },
  { name: "Grace", score: 85 },
  { name: "Linus", score: 70 }
];

const reportLines = students.map(student => {
  return `${student.name}: ${student.score}`;
});

console.log(reportLines);
```

Expected output:

```
["Ada: 90", "Grace: 85", "Linus: 70"]
```

16. Common `map()` Mistake: Forgetting `return`

This is wrong:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => {
  number * 2;
});

console.log(doubled);
```

Expected output:

```
[undefined, undefined, undefined]
```

Why?

Because this callback uses curly braces:

```
number => {
  number * 2;
}
```

But it does not return anything.

So each callback call returns `undefined`.

Correct version with explicit return:

```
const doubled = numbers.map(number => {
  return number * 2;
});
```

Correct version with implicit return:

```
const doubled = numbers.map(number => number * 2);
```

Beginner rule:

If your `map()` output is full of `undefined`, check whether you forgot `return`.

17. `filter()`

Use `filter()` when you want to keep only some items.

Simple meaning:

Test each item. Keep the item if the test passes.

Example:

```
const scores = [90, 45, 80, 60, 100];

const passingScores = scores.filter(score => {
  return score >= 70;
});

console.log(passingScores);
```

Expected output:

```
[90, 80, 100]
```

Shorter version:

```
const passingScores = scores.filter(score => score >= 70);
```

18. How `filter()` Decides What to Keep

The callback must return a truthy or falsy value.

Usually, it returns a boolean:

true

means:

Keep this item.

false

means:

Remove this item.

Example:

```
const scores = [90, 45, 80];  
  
const passingScores = scores.filter(score => score >= 70);  
  
console.log(passingScores);
```

Dry run:

Score	Test <code>score >= 70</code>	Keep?
90	true	Yes
45	false	No
80	true	Yes

Final result:

[90, 80]

Beginner rule:

Use `filter()` when the output may have fewer items than the input.

19. `filter()` With Objects

Example:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const expensiveProducts = products.filter(product => product.price >= 10);

console.log(expensiveProducts);
```

Expected output:

```
[
  { name: "Book", price: 12 },
  { name: "Bag", price: 30 }
]
```

What happened?

- Book costs 12, so it is kept.
- Pen costs 2, so it is removed.
- Bag costs 30, so it is kept.

20. `filter()` vs `map()`

These methods have different jobs.

Method	Main Question	Result
<code>map()</code>	How should each item change?	New array with transformed items.
<code>filter()</code>	Should this item stay?	New array with only matching items.

Example with `map()`:

```
const numbers = [1, 2, 3];
const doubled = numbers.map(number => number * 2);
console.log(doubled);
```

Expected output:

```
[2, 4, 6]
```

Example with `filter()`:

```
const numbers = [1, 2, 3];  
const biggerThanOne = numbers.filter(number => number > 1);  
console.log(biggerThanOne);
```

Expected output:

```
[2, 3]
```

Simple memory:

- `map()` transforms.
- `filter()` selects.

21. `reduce()`

Use `reduce()` when you want to combine many values into one result.

Simple meaning:

Reduce an array down to one final value.

Common uses:

- total
- average
- count
- maximum or minimum
- object summary
- grouped data
- one combined string

Basic example:

```
const numbers = [1, 2, 3, 4];  
  
const total = numbers.reduce((sum, number) => {  
  return sum + number;  
});
```

```
}, 0);  
  
console.log(total);
```

Expected output:

10

22. Parts of `reduce()`

Look at this code:

```
const total = numbers.reduce((sum, number) => {  
  return sum + number;  
}, 0);
```

Important parts:

Part	Meaning
<code>sum</code>	The accumulator. It stores the running result.
<code>number</code>	The current item.
<code>return sum + number</code>	The new accumulator value.
<code>0</code>	The starting value of the accumulator.

The accumulator is the value that carries through the whole process.

In this example, the accumulator is called `sum` because it stores the running total.

23. `reduce()` Dry Run

Code:

```
const numbers = [1, 2, 3, 4];  
  
const total = numbers.reduce((sum, number) => {
```

```
    return sum + number;  
  }, 0);
```

Dry run:

Step	sum Before	number	Returned Value	sum After
Start	0	-	-	0
1	0	1	1	1
2	1	2	3	3
3	3	3	6	6
4	6	4	10	10

Final result:

10

Simple explanation:

```
0 + 1 = 1  
1 + 2 = 3  
3 + 3 = 6  
6 + 4 = 10
```

So `total` becomes:

10

24. Why the Initial Value Matters in `reduce()`

This part is the initial value:

0

Example:


```
const total = numbers.reduce((sum, number) => {  
  return sum + number;  
}, 0);
```

Using an initial value makes your code clearer and safer.

For totals, use:

```
0
```

For building an array, use:

```
[]
```

For building an object, use:

```
{}
```

Beginner rule:

Give `reduce()` an initial value unless you have a specific reason not to.

25. `reduce()` With Objects

Example:

```
const products = [  
  { name: "Book", price: 12 },  
  { name: "Pen", price: 2 },  
  { name: "Bag", price: 30 }  
];  
  
const totalPrice = products.reduce((sum, product) => {  
  return sum + product.price;  
}, 0);  
  
console.log(totalPrice);
```

Expected output:

44

Dry run:

Product	Price	Running Sum
Start	-	0
Book	12	12
Pen	2	14
Bag	30	44

Final result:

44

26. `reduce()` for Counting

You can use `reduce()` to count items that match a condition.

Example:

```
const scores = [90, 45, 80, 60, 100];

const passingCount = scores.reduce((count, score) => {
  if (score >= 70) {
    return count + 1;
  }

  return count;
}, 0);

console.log(passingCount);
```

Expected output:

3

What happened?

- 90 passes, count becomes 1.
- 45 does not pass, count stays 1.
- 80 passes, count becomes 2.
- 60 does not pass, count stays 2.
- 100 passes, count becomes 3.

Final result:

3

27. `reduce()` vs `map()` vs `filter()`

Method	Use It When	Result
<code>map()</code>	You want to transform each item.	New array.
<code>filter()</code>	You want to keep only matching items.	New array.
<code>reduce()</code>	You want one final result.	Any value.

Example data:

```
const numbers = [1, 2, 3, 4];
```

Transform with `map()`:

```
const doubled = numbers.map(number => number * 2);  
console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

Select with `filter()`:

```
const evens = numbers.filter(number => number % 2 === 0);  
console.log(evens);
```

Output:

```
[2, 4]
```

Combine with `reduce()`:

```
const total = numbers.reduce((sum, number) => sum + number, 0);
console.log(total);
```

Output:

```
10
```

28. Method Chaining

Method chaining means calling one method after another.

Example:

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers
  .filter(number => number % 2 === 1)
  .map(number => number * 2)
  .reduce((total, number) => total + number, 0);

console.log(result);
```

Expected output:

```
18
```

Read chained methods from top to bottom.

29. Method Chaining Step-by-Step

Start:

```
[1, 2, 3, 4, 5]
```

Step 1:

```
.filter(number => number % 2 === 1)
```

This keeps odd numbers.

Result:

```
[1, 3, 5]
```

Step 2:

```
.map(number => number * 2)
```

This doubles each remaining number.

Result:

```
[2, 6, 10]
```

Step 3:

```
.reduce((total, number) => total + number, 0)
```

This adds the numbers together.

Result:

```
18
```

Final output:

```
18
```

Beginner rule:

Each method receives the result of the method before it.

30. Chaining With Objects

Example:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);

console.log(total);
```

Expected output:

42

Step-by-step:

Start:

```
[
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
]
```

After `filter()`:

```
[
  { name: "Book", price: 12 },
  { name: "Bag", price: 30 }
]
```

After `map()`:

[12, 30]

After `reduce()`:

42

Simple meaning:

Keep products priced at least 10, extract their prices, then add those prices.

31. When Chaining Is Useful

Chaining is useful when each step has a clear job.

Good chain:

```
const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);
```

This is readable because each line does one thing:

1. Select products.
2. Extract prices.
3. Add prices.

Weak chain:

```
const result = products.filter(p => p.price >= 10).map(p => p.price).reduce((s,
p) => s + p, 0);
```

This works, but it is harder to read.

Beginner rule:

Put each chained method on its own line when the chain gets long.

32. `sort()`

Use `sort()` to reorder an array.

Example:

```
const names = ["Grace", "Ada", "Linus"];  
  
names.sort();  
  
console.log(names);
```

Expected output:

```
["Ada", "Grace", "Linus"]
```

For strings, the default sort can often look correct.

For numbers, there is a trap.

33. Default `sort()` Treats Values Like Strings

Example:

```
const scores = [414, 200, 5];  
  
scores.sort();  
  
console.log(scores);
```

Expected output:

```
[200, 414, 5]
```

This looks wrong if you expect numeric order.

Why does this happen?

Because default `sort()` compares values as strings.

So JavaScript compares something like:

```
"200"  
"414"  
"5"
```

As text, `"5"` can come after `"414"`.

Beginner rule:

Always use a compare function when sorting numbers.

34. Numeric Sort: Ascending Order

Ascending means smallest to largest.

Example:

```
const scores = [414, 200, 5];  
  
scores.sort((a, b) => a - b);  
  
console.log(scores);
```

Expected output:

```
[5, 200, 414]
```

The compare function is:

```
(a, b) => a - b
```

Simple meaning:

Sort numbers from low to high.

35. Numeric Sort: Descending Order

Descending means largest to smallest.

Example:

```
const scores = [414, 200, 5];  
  
scores.sort((a, b) => b - a);  
  
console.log(scores);
```

Expected output:

```
[414, 200, 5]
```

The compare function is:

```
(a, b) => b - a
```

Simple meaning:

Sort numbers from high to low.

36. How Compare Functions Work

A compare function receives two values:

```
(a, b) => a - b
```

The result tells JavaScript how to order them.

Return Value	Meaning
Negative	Put <code>a</code> before <code>b</code> .
Positive	Put <code>b</code> before <code>a</code> .
0	Keep their order relative to each other.

Beginner practical rule:

```
(a, b) => a - b
```

means ascending numbers.

```
(a, b) => b - a
```

means descending numbers.

You do not need to memorize advanced sorting theory at first. Just remember these two patterns.

37. `sort()` Mutates the Original Array

`sort()` changes the original array.

Example:

```
const scores = [414, 200, 5];  
  
scores.sort((a, b) => a - b);  
  
console.log(scores);
```

Expected output:

```
[5, 200, 414]
```

The original `scores` array was changed.

This is different from `map()` and `filter()`, which return new arrays.

Beginner warning:

`sort()` mutates. Be careful when you need to preserve the original array.

38. Copy Before Sorting

If you want to keep the original array unchanged, make a copy first.

Example:

```
const scores = [414, 200, 5];

const sortedScores = [...scores].sort((a, b) => a - b);

console.log(scores);
console.log(sortedScores);
```

Expected output:

```
[414, 200, 5]
[5, 200, 414]
```

This part copies the array:

```
[...scores]
```

Then this sorts the copy:

```
.sort((a, b) => a - b)
```

Simple meaning:

Copy first, then sort the copy.

39. Sorting Objects

You can sort arrays of objects by a property.

Example:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const sortedProducts = [...products].sort((a, b) => a.price - b.price);

console.log(sortedProducts);
```

Expected output:

```
[
  { name: "Pen", price: 2 },
  { name: "Book", price: 12 },
  { name: "Bag", price: 30 }
]
```

This sorts by `price` from lowest to highest.

Descending by price:

```
const sortedProducts = [...products].sort((a, b) => b.price - a.price);
```

40. `every()`

Use `every()` when all items must pass a test.

Simple meaning:

Does every item pass this condition?

Example:

```
const scores = [80, 90, 100];

const allPassing = scores.every(score => score >= 70);

console.log(allPassing);
```

Expected output:

```
true
```

Why?

Every score is at least 70.

Example where one item fails:

```
const scores = [80, 50, 100];  
  
const allPassing = scores.every(score => score >= 70);  
  
console.log(allPassing);
```

Expected output:

```
false
```

Why?

The score `50` does not pass.

Beginner rule:

`every()` returns `true` only if all items pass.

41. `some()`

Use `some()` when at least one item must pass a test.

Simple meaning:

Does at least one item pass this condition?

Example:

```
const scores = [40, 50, 90];  
  
const hasPassingScore = scores.some(score => score >= 70);  
  
console.log(hasPassingScore);
```

Expected output:

```
true
```

Why?

At least one score is at least 70.

Example where no item passes:

```
const scores = [40, 50, 60];

const hasPassingScore = scores.some(score => score >= 70);

console.log(hasPassingScore);
```

Expected output:

```
false
```

Why?

No score is at least 70.

Beginner rule:

`some()` returns `true` if one or more items pass.

42. `every()` vs `some()`

Method	Question	Returns <code>true</code> When
<code>every()</code>	Do all items pass?	Every item passes.
<code>some()</code>	Does at least one item pass?	One or more items pass.

Example:

```
const scores = [80, 50, 100];

console.log(scores.every(score => score >= 70));
console.log(scores.some(score => score >= 70));
```

Expected output:

```
false  
true
```

Why?

- `every()` is false because 50 fails.
- `some()` is true because 80 and 100 pass.

Simple memory:

- `every()` means all.
- `some()` means at least one.

43. Side Effects vs Returned Values

This distinction is critical in Sprint 9.

A side effect means the code does something outside itself.

Examples of side effects:

- `console.log()`
- changing the page
- changing an outside variable
- saving data
- showing an alert

A returned value means a function sends a value back.

Example with side effect:

```
function showTotal(total) {  
  console.log(total);  
}
```

This prints the total, but it does not return it.

Example with returned value:


```
function getTotal(a, b) {  
  return a + b;  
}
```

This gives a reusable result.

Sprint 9 rule:

Method	Usually Used For
<code>forEach()</code>	Side effects.
<code>map()</code>	Returned transformed values.
<code>filter()</code>	Returned selected values.
<code>reduce()</code>	Returned final result.

44. `forEach()` vs `map()`

A common beginner mistake is using `forEach()` when `map()` is needed.

Wrong if you need a new array:

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.forEach(number => {  
  return number * 2;  
});  
  
console.log(doubled);
```

Expected output:

```
undefined
```

Correct:

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(number => number * 2);
```

```
console.log(doubled);
```

Expected output:

```
[2, 4, 6]
```

Beginner rule:

If you need a new array, use `map()` or `filter()`, not `forEach()`.

45. Main Array Method Summary

Method	Main Job	Callback Should Return	Method Returns	Changes Original Array?
<code>forEach()</code>	Do something for each item.	Usually nothing important.	<code>undefined</code>	No, unless your callback changes something.
<code>map()</code>	Transform each item.	New value for each item.	New array.	No.
<code>filter()</code>	Keep matching items.	Boolean/truthy-falsy test.	New array.	No.
<code>reduce()</code>	Combine into one result.	New accumulator value.	Final accumulator.	No, unless your callback changes something.
<code>sort()</code>	Reorder items.	Compare number.	Same array reference.	Yes.
<code>every()</code>	Check if all pass.	Boolean/truthy-falsy test.	Boolean.	No.
<code>some()</code>	Check if at least one passes.	Boolean/truthy-falsy test.	Boolean.	No.

46. Choosing the Right Method

Use this guide.

Use `forEach()` when you want to do an action for every item.

Example:

```
names.forEach(name => console.log(name));
```

Question:

Do I want to do something with each item, but I do not need a new array?

Use `forEach()`.

Use `map()` when you want to transform every item.

Example:

```
const doubled = numbers.map(number => number * 2);
```

Question:

Do I want one new item for every original item?

Use `map()`.

Use `filter()` when you want to select some items.

Example:

```
const passing = scores.filter(score => score >= 70);
```

Question:

Do I want to keep only items that pass a condition?

Use `filter()`.

Use `reduce()` when you want one final result.

Example:

```
const total = scores.reduce((sum, score) => sum + score, 0);
```

Question:

Do I want to combine the array into one value?

Use `reduce()`.

Use `sort()` when you want to reorder items.

Example:

```
const sorted = [...scores].sort((a, b) => a - b);
```

Question:

Do I want the items in a different order?

Use `sort()`.

Use `every()` when all items must pass.

Example:

```
const allPassing = scores.every(score => score >= 70);
```

Question:

Do all items pass this condition?

Use `every()`.

Use `some()` when at least one item must pass.

Example:

```
const hasPerfectScore = scores.some(score => score === 100);
```

Question:

Does at least one item pass this condition?

Use `some()`.

47. Core Code Pattern

Type this code yourself and predict the output before running it.

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers
  .filter(number => number % 2 === 1)
  .map(number => number * 2)
  .reduce((total, number) => total + number, 0);

console.log(result);
```

Expected output:

18

Explanation:

Start:

[1, 2, 3, 4, 5]

Keep odd numbers:

[1, 3, 5]

Double them:

[2, 6, 10]

Add them:

18

48. Mini-Build: Grade Report

This mini-build uses several Sprint 9 methods together.

```
const scores = [90, 45, 80, 60, 100];

const passingScores = scores.filter(score => score >= 70);

const sortedScores = [...scores].sort((a, b) => a - b);

const total = scores.reduce((sum, score) => sum + score, 0);

const average = total / scores.length;

const everyonePassed = scores.every(score => score >= 70);

const hasPerfectScore = scores.some(score => score === 100);

console.log("Passing scores:", passingScores);
console.log("Sorted scores:", sortedScores);
console.log("Average:", average);
console.log("Everyone passed:", everyonePassed);
console.log("Has perfect score:", hasPerfectScore);
```

Expected output:

```
Passing scores: [90, 80, 100]
Sorted scores: [45, 60, 80, 90, 100]
Average: 75
Everyone passed: false
Has perfect score: true
```

49. Grade Report Explanation

Original array:

```
[90, 45, 80, 60, 100]
```

Passing scores

```
const passingScores = scores.filter(score => score >= 70);
```

Keeps:

```
[90, 80, 100]
```

Sorted scores

```
const sortedScores = [...scores].sort((a, b) => a - b);
```

Copies first, then sorts from lowest to highest:

```
[45, 60, 80, 90, 100]
```

The original array stays unchanged because of:

```
[...scores]
```

Total

```
const total = scores.reduce((sum, score) => sum + score, 0);
```

Calculates:

```
90 + 45 + 80 + 60 + 100 = 375
```

Average

```
const average = total / scores.length;
```

Calculates:

```
375 / 5 = 75
```

Everyone passed

```
const everyonePassed = scores.every(score => score >= 70);
```

Returns:

```
false
```

because 45 and 60 are below 70.

Has perfect score

```
const hasPerfectScore = scores.some(score => score === 100);
```

Returns:

```
true
```

because one score is exactly 100.

50. Mini-Build: Product Total From Filtered Products

This example combines objects with Sprint 9 array methods.

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);

console.log(total);
```

Expected output:

42

Step-by-step:

Start:

```
[  
  { name: "Book", price: 12 },  
  { name: "Pen", price: 2 },  
  { name: "Bag", price: 30 }  
]
```

After `filter()`:

```
[  
  { name: "Book", price: 12 },  
  { name: "Bag", price: 30 }  
]
```

After `map()`:

[12, 30]

After `reduce()`:

42

51. Common Beginner Mistakes

Mistake 1: Calling the callback immediately

Wrong:

```
runTask(sayHello());
```

Correct:

```
runTask(sayHello);
```

Why?

You usually pass the function itself, not the result of calling it.

Mistake 2: Forgetting `return` in `map()`

Wrong:

```
const doubled = numbers.map(number => {  
  number * 2;  
});
```

Correct:

```
const doubled = numbers.map(number => {  
  return number * 2;  
});
```

Also correct:

```
const doubled = numbers.map(number => number * 2);
```

Mistake 3: Using `forEach()` to create a new array

Wrong:

```
const doubled = numbers.forEach(number => number * 2);
```

Correct:

```
const doubled = numbers.map(number => number * 2);
```

Why?

`forEach()` returns `undefined`.

`map()` returns a new array.

Mistake 4: Sorting numbers without a compare function

Wrong:

```
const scores = [414, 200, 5];
scores.sort();
console.log(scores);
```

Output:

```
[200, 414, 5]
```

Correct:

```
scores.sort((a, b) => a - b);
```

Output:

```
[5, 200, 414]
```

Mistake 5: Forgetting that `sort()` changes the original array

Risky:

```
const scores = [414, 200, 5];
const sorted = scores.sort((a, b) => a - b);
```

Safer:

```
const scores = [414, 200, 5];
const sorted = [...scores].sort((a, b) => a - b);
```

Mistake 6: Using `every()` when you mean `some()`

Wrong if you only need one passing score:

```
const hasPassingScore = scores.every(score => score >= 70);
```

Correct:

```
const hasPassingScore = scores.some(score => score >= 70);
```

Use `every()` for all.

Use `some()` for at least one.

Mistake 7: Making chains too hard to read

Harder to read:

```
const total = products.filter(p => p.price >= 10).map(p => p.price).reduce((s, p) => s + p, 0);
```

Clearer:

```
const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);
```

52. Debug Challenge 1: Fix the `map()` Return

Broken code:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => {
  number * 2;
});
```

```
console.log(doubled);
```

Current output:

```
[undefined, undefined, undefined]
```

Problem:

The callback uses curly braces but does not return anything.

Fixed version:

```
const numbers = [1, 2, 3];

const doubled = numbers.map(number => {
  return number * 2;
});

console.log(doubled);
```

Expected output:

```
[2, 4, 6]
```

Short fixed version:

```
const doubled = numbers.map(number => number * 2);
```

53. Debug Challenge 2: Fix Numeric Sorting

Broken code:

```
const scores = [414, 200, 5];

scores.sort();

console.log(scores);
```

Problem:

Default `sort()` compares values like strings.

Fixed version:

```
const scores = [414, 200, 5];  
  
scores.sort((a, b) => a - b);  
  
console.log(scores);
```

Expected output:

```
[5, 200, 414]
```

Safer version that protects the original:

```
const scores = [414, 200, 5];  
  
const sortedScores = [...scores].sort((a, b) => a - b);  
  
console.log(scores);  
console.log(sortedScores);
```

Expected output:

```
[414, 200, 5]  
[5, 200, 414]
```

54. Debug Challenge 3: Pick the Correct Method

Problem:

You want this:

```
[1, 2, 3]
```

to become this:

```
[2, 4, 6]
```

Wrong method:

```
const result = numbers.forEach(number => number * 2);
```

Correct method:

```
const result = numbers.map(number => number * 2);
```

Reason:

You need a new transformed array, so use `map()`.

55. Practice Task 1: Print Names

Data:

```
const names = ["Ada", "Grace", "Linus"];
```

Task:

Use `forEach()` to print every name.

Solution:

```
const names = ["Ada", "Grace", "Linus"];

names.forEach(name => {
  console.log(name);
});
```

Expected output:

```
Ada
Grace
Linus
```

56. Practice Task 2: Double Numbers

Data:

```
const numbers = [1, 2, 3, 4];
```

Task:

Use `map()` to create:

```
[2, 4, 6, 8]
```

Solution:

```
const numbers = [1, 2, 3, 4];  
  
const doubled = numbers.map(number => number * 2);  
  
console.log(doubled);
```

Expected output:

```
[2, 4, 6, 8]
```

57. Practice Task 3: Keep Passing Scores

Data:

```
const scores = [90, 45, 80, 60, 100];
```

Task:

Use `filter()` to keep scores that are at least 70.

Solution:

```
const scores = [90, 45, 80, 60, 100];
```



```
const passingScores = scores.filter(score => score >= 70);  
  
console.log(passingScores);
```

Expected output:

```
[90, 80, 100]
```

58. Practice Task 4: Calculate Total

Data:

```
const prices = [12, 2, 30];
```

Task:

Use `reduce()` to calculate the total.

Solution:

```
const prices = [12, 2, 30];  
  
const total = prices.reduce((sum, price) => sum + price, 0);  
  
console.log(total);
```

Expected output:

```
44
```

59. Practice Task 5: Check All Passing

Data:

```
const scores = [80, 90, 100];
```

Task:

Use `every()` to check whether all scores are at least 70.

Solution:

```
const scores = [80, 90, 100];

const allPassing = scores.every(score => score >= 70);

console.log(allPassing);
```

Expected output:

```
true
```

60. Practice Task 6: Check If Any Perfect Score Exists

Data:

```
const scores = [80, 90, 100];
```

Task:

Use `some()` to check whether at least one score is 100.

Solution:

```
const scores = [80, 90, 100];

const hasPerfectScore = scores.some(score => score === 100);

console.log(hasPerfectScore);
```

Expected output:

```
true
```

61. Practice Task 7: Sort Scores Safely

Data:

```
const scores = [414, 200, 5];
```

Task:

Create a sorted copy from lowest to highest.

Solution:

```
const scores = [414, 200, 5];

const sortedScores = [...scores].sort((a, b) => a - b);

console.log(scores);
console.log(sortedScores);
```

Expected output:

```
[414, 200, 5]
[5, 200, 414]
```

62. Practice Task 8: Chain Methods

Data:

```
const numbers = [1, 2, 3, 4, 5];
```

Task:

1. Keep odd numbers.
2. Double them.
3. Add them.

Solution:

```
const numbers = [1, 2, 3, 4, 5];
```

```
const result = numbers
  .filter(number => number % 2 === 1)
  .map(number => number * 2)
  .reduce((total, number) => total + number, 0);

console.log(result);
```

Expected output:

18

63. Sprint 9 Mental Models

Callback

A function passed into another function.

Higher-order function

A function that takes a function or returns a function.

forEach()

Do something for each item.

map()

Transform each item into a new item.

filter()

Keep only items that pass a test.

reduce()

Combine all items into one result.

sort()

Reorder items, but remember it changes the original array.

`every()`

Check whether all items pass.

`some()`

Check whether at least one item passes.

Method chaining

Use the result of one method as the input to the next method.

64. Sprint 9 Revision Checkpoint

Answer these without looking at the notes.

1. What is a callback function?
2. What is a higher-order function?
3. What is the difference between `sayHello` and `sayHello()`?
4. What does `forEach()` return?
5. When should you use `forEach()`?
6. What does `map()` return?
7. When should you use `map()`?
8. What does `filter()` return?
9. What should a `filter()` callback return?
10. What is the accumulator in `reduce()`?
11. Why is the initial value in `reduce()` useful?
12. What is method chaining?
13. Why does default `sort()` give surprising results with numbers?
14. How do you sort numbers from lowest to highest?
15. How do you sort numbers from highest to lowest?
16. Why should you copy an array before sorting if you need to preserve the original?
17. What does `every()` check?
18. What does `some()` check?
19. Why does this produce `undefined` values?

```
const doubled = numbers.map(number => {  
  number * 2;  
});
```

1. Which method would you use to calculate the total price of products?
-

65. Sprint 9 Minimum Passing Standard

You are ready for Sprint 10 when you can explain this code line by line:

```
const products = [
  { name: "Book", price: 12 },
  { name: "Pen", price: 2 },
  { name: "Bag", price: 30 }
];

const total = products
  .filter(product => product.price >= 10)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);

console.log(total);
```

Expected output:

42

You should be able to say:

1. `filter()` keeps products with price at least 10.
2. `map()` extracts the prices from those products.
3. `reduce()` adds those prices together.
4. The final result is `42`.

66. Sprint 9 Final Summary

Sprint 9 is about using functions to process arrays clearly.

The biggest shift is:

Instead of manually writing every loop, you choose the array method that matches your goal.

Use this final guide:

Goal	Method
Do something for each item	<code>forEach()</code>
Transform every item	<code>map()</code>

Goal	Method
Keep only matching items	<code>filter()</code>
Combine into one result	<code>reduce()</code>
Reorder items	<code>sort()</code>
Check if all items pass	<code>every()</code>
Check if at least one item passes	<code>some()</code>

Main mental model:

Loops say: "Go through the array manually."

Sprint 9 methods say: "Tell JavaScript what kind of processing you want."

Once you understand Sprint 9, you can write cleaner code for lists of numbers, strings, objects, products, scores, users, tasks, and many other collections.